DURGA

SYSTEM AND DEVELOPMENT DOCUMENTATION

Frode Randers

2026-04-04

Contents

1 Introduction

Durga is a BPMN-driven Kafka scaffolding tool. It reads BPMN models, extracts an executable subset of process semantics, and generates Java and Kafka-oriented project skeletons that can be used to explore, test and refine workflow designs.

The project is not a finished workflow engine. It is best understood as a generator plus a local experimentation surface:

- BPMN models are parsed with Camunda,
- Java runtime skeletons are rendered with StringTemplate,
- Kafka topics and reactive-messaging bindings are generated from the process graph,
- supporting probes and demos are generated so the resulting process can be exercised locally,
- a separate Kafka Streams monitoring path materializes process state and operational views.

This manual describes the current system shape rather than an ideal future architecture. It focuses on what Durga generates today, what runtime model those generated artifacts implement, and where the present support boundary still ends.

The document is organized in that order. Section 2 describes the major subsystems. Section 4 explains the BPMN-to-Kafka execution mapping. Section 6 describes the canonical process-event and monitoring model. Section 9 describes the generated project surface, and Section 21 summarizes maturity, constraints and current gaps.

2 System Overview

At a high level, Durga has three centers:

- a BPMN parsing and generation pipeline,
- a generated Kafka-oriented process runtime,
- a monitoring and reporting path built around canonical process lifecycle events.

2.1 Repository-level components

The repository contains both the generator and reusable runtime support:

Scaffolder The main entry point parses a BPMN model and renders a generated project. The orchestration entry point is `tools.org.gautelis.durga.BpmnScaffolder`.
Generation support Model extraction and template coordination are factored across helper classes such as:

- `BpmnModelSupport`,
- `TaskRoutingGenerator`,
- `EventTemplateGenerator`,
- `SubProcessTemplateGenerator`,
- `GeneratedProjectSupport`.

Templates StringTemplate groups under `src/main/resources/templates/` define the generated Java classes, scripts, YAML, Maven build files and shared runtime helpers.
Monitoring runtime A separate Kafka Streams topology consumes canonical process lifecycle events and materializes state, counts, latency summaries and stuck-instance views.

2.2 Generated runtime shape

For each BPMN process, Durga generates a small standalone project with:

- worker and routing services,
- a starter,
- a process orchestrator,
- explicit helpers for user/manual task completion and failure testing,
- generated Kafka topic configuration,
- sample payloads and runnable scenario scripts.

The runtime style is pragmatic rather than minimal. Topics and handlers are made deliberately visible so that users can inspect how BPMN concepts are projected onto Kafka channels and lifecycle events.

2.3 Canonical event model

The generated execution topics are not the only runtime surface. Durga also emits a shared canonical lifecycle stream on `process-events`. That stream is the observability backbone for:

- latest state per process instance,

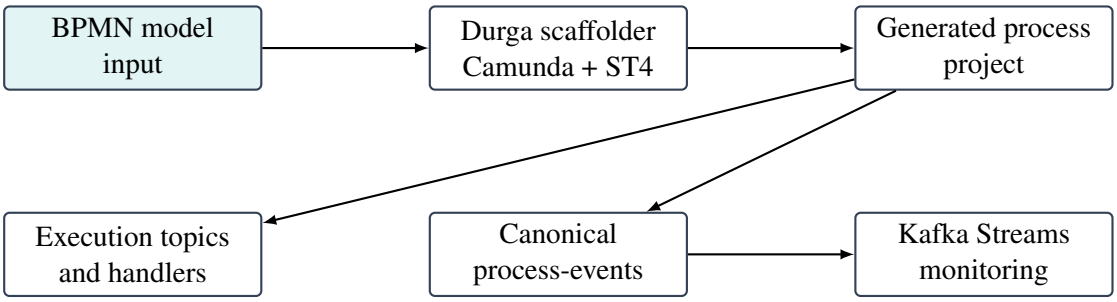

Figure 1: High-level Durga flow from BPMN model to generated runtime and monitoring projections

- counts by current state,
- latency summaries,
- stuck-instance detection,
- dashboards and query endpoints.

This split is deliberate. Topic-per-task or topic-per-edge wiring may be useful for execution visibility, but the monitoring model depends on explicit projections rather than on raw topic inspection.

3 Architecture Decisions

Durga is not only a generator. It is a generator with a fairly opinionated runtime shape. Several choices were made deliberately because they improve inspectability and make the system easier to discuss, test and evolve.

3.1 Canonical lifecycle stream

The most important decision is the shared `process-events` topic. Execution topics carry work between generated handlers, but they are not treated as the authoritative query surface. Durga instead emits normalized lifecycle events and builds state projections from that stream.

This was chosen because:

- execution topics represent transport history, not current truth,
- process instance state must be queryable independently of routing topology,
- counts, latency summaries and stuck-instance views are easier to derive from explicit lifecycle events than from task-topic inspection.

3.2 Visible generated runtime

Durga generates many explicit classes instead of one hidden execution engine. That is a tradeoff: the output is more verbose, but it is also far easier to inspect. This supports the project goal of making BPMN-to-Kafka mapping concrete rather than magical.

3.3 Generated helper scripts as product surface

The helper scripts are deliberate, not incidental. A BPMN scaffolding tool is difficult to evaluate if the user must write custom producers and consumers before seeing anything happen. Durga therefore generates probes, scenario runners and manual interaction scripts together with the runtime.

3.4 Kafka Streams as the default read model

The monitoring side uses Kafka Streams because it fits the core question well: consume one canonical stream and materialize queryable state from it. The current implementation is not trying to solve every production reporting problem; it is solving the local and architectural read-model problem directly.

3.5 Scope-aware subprocess handling

Subprocess support could have been left as simple flattening. Durga started that way, but then moved toward explicit subprocess entry and completion services because scope behavior matters once cancellation, failure, escalation and event subprocesses are introduced.

4 Execution Model

4.1 General mapping strategy

Durga maps a BPMN graph into a bounded Kafka runtime surface. In the current implementation, each supported BPMN element is converted into either:

- a generated service consuming from one topic and producing to one or more topics,
- a scope handler that turns subprocess semantics into lifecycle and routing events,
- or an external helper path that lets a human or another tool supply the missing runtime action.

The model is intentionally explicit. Instead of hiding process behavior behind one opaque runtime, the generated project exposes the process topology as concrete Kafka topics, generated services and helper scripts.

4.2 Core control-flow support

The executable subset currently includes:

- start events,
- service, user and manual tasks,
- exclusive and parallel gateways,
- call activities,
- explicit end-event completion,
- embedded subprocesses,
- several categories of boundary events,
- event subprocesses for selected start types.

The important distinction is that not all BPMN elements are treated the same way. A service task auto-completes in the generated worker model, while a user or manual task enters a waiting state and must be advanced through a generated completion helper.

4.3 Tasks and routing

Service tasks are generated as workers consuming `%processId%_%taskId%_input` and producing `%processId%_%taskId%_output`.

User and manual tasks consume their input topic, emit an `ACTIVITY_ENTERED` lifecycle event, and then wait for an external completion event generated via helper scripts.

Exclusive gateways are generated as routing services that evaluate simple conditions against the event payload and emit the selected next activity. Parallel splits fan out to several downstream inputs. Parallel joins use the generated process-state store to wait for the required set of incoming completions before forwarding the flow.

4.4 Boundary behavior

Boundary timer, error and escalation handlers are generated from BPMN boundary events and consume the canonical lifecycle stream on `process-events-monitor`. The current boundary implementation supports:

- interrupting and non-interrupting timer boundaries,
- interrupting and non-interrupting error boundaries,
- interrupting and non-interrupting escalation boundaries,
- task-attached and subprocess-attached variants.

Interrupting boundaries now have behavioral effect, not just monitoring effect. Generated handlers consult the shared cancellation registry so that canceled work is suppressed rather than continuing to emit normal downstream progress.

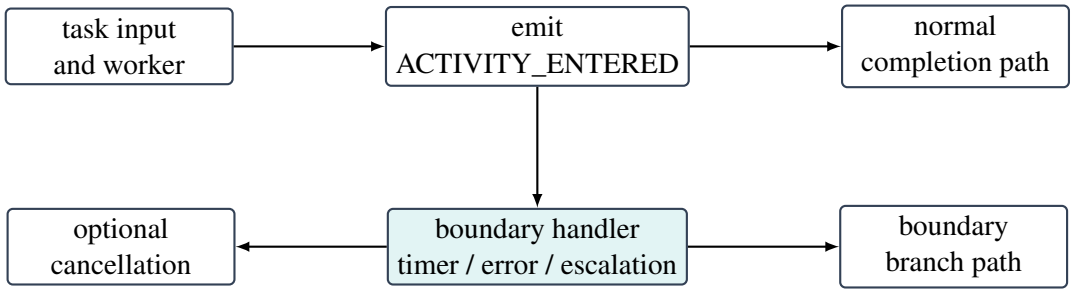

Figure 2: Generated boundary handling separates normal flow from boundary-triggered flow

4.5 Subprocesses and event subprocesses

Embedded subprocesses are generated with explicit entry and completion services. These services emit subprocess-level lifecycle events and route into and out of the internal subprocess graph. Nested subprocesses are supported with the same pattern.

Event subprocesses are currently implemented as scoped handlers rather than as a full general-purpose BPMN scope engine. The supported start types are:

- message,
- signal,
- timer inside an embedded subprocess scope,
- error inside an embedded subprocess scope,
- escalation inside an embedded subprocess scope.

Message and signal event subprocesses consume from dedicated external topics. Timer, error and escalation event subprocesses instead listen to scoped lifecycle signals on `process-events-monitor`. Interrupting event subprocesses emit cancellation for the enclosing scope before starting the event subprocess branch. Non-interrupting event subprocesses branch without canceling the parent flow.

4.6 Current execution boundary

The current execution boundary is broad enough for local process experimentation, but it is not full BPMN coverage. Inclusive gateways, complex gateways, BPMN-native data objects, and more advanced scope semantics are still out of scope. That boundary should be treated as a deliberate implementation limit, not as an accidental omission.

5 BPMN Support And Limits

Durga supports a pragmatic BPMN subset that is broad enough for exploration and code generation, but not yet a complete BPMN execution engine.

5.1 Well-supported areas

The strongest current coverage includes:

- start events and explicit end completion handling,
- service, user and manual task mappings,
- XOR and parallel gateway routing,
- intermediate timer, message and signal events,
- task and subprocess boundary timer, error and escalation handling,
- call activities,
- embedded subprocesses including nested subprocesses,
- event subprocesses for message, signal, timer, error and escalation starts.

5.2 Current semantic boundaries

Some behaviors are supported only to the level needed for explicit generated runtimes, not to full BPMN-engine fidelity:

- cancellation propagation is pragmatic rather than globally token-precise,
- subprocess scope handling is explicit but still lighter than a full BPMN execution scope manager,
- monitoring is strong on current state, lighter on long-term history and trend analysis,
- generated runtimes are optimized for understanding and experimentation rather than minimalism.

5.3 Still outside the current focus

The main areas still outside the project's current center of gravity are:

- full production-grade workflow-engine semantics,
- richer compensation and advanced BPMN exception orchestration,
- large-scale operational hardening beyond local and exploratory usage,
- a generalized multi-node monitoring and query deployment model.

The repository coverage matrix remains the most detailed element-by-element reference, but the overall design intent is simple: support enough BPMN to make the Kafka mapping useful, while keeping the generated runtime explicit and teachable.

6 Monitoring and Observability

6.1 Canonical process-events topic

Durga emits a normalized lifecycle event stream on the shared topic `process-events`. This stream is intended to be the observability backbone of the system. Generated workers, gateways, subprocess handlers, boundary handlers and helper publishers all emit lifecycle events describing process progress, waiting states, completion, failure, escalation and cancellation.

This design separates execution transport from queryable state. The generated execution topics remain useful for routing and local inspection, but the monitoring model depends on projections built from explicit lifecycle events rather than on direct counting of messages in task topics.

6.2 Kafka Streams topology

The repository contains a Kafka Streams monitoring topology that consumes `process-events` and materializes:

- latest process state by instance,
- counts by current state,
- latency summaries,
- stuck-instance views.

The main entry point is `org.gautelis.durga.monitoring.ProcessMonitoringApp`. The projected instance model is represented by `ProcessStateView`. The query side is exposed both through a small HTTP service and through a CLI client.

6.3 Query surface

The monitoring service exposes endpoints for health, instance lookup, counts, latency and stuck instances. A lightweight dashboard is also provided. This gives Durga a useful local loop:

1. generate a process,
2. publish scenarios or task inputs,
3. observe lifecycle events,
4. query current state and latency,
5. iterate on the BPMN model or generated runtime.

6.4 Operational significance

This monitoring path matters because Durga is not just a code generator. It is also a learning and exploration surface. The generated helpers and the Streams-based monitoring path make it possible to understand how process instances move through the generated graph, which states accumulate, where latency emerges, and where cancellation or timeout behavior changes the normal flow.

7 Monitoring Topology

7.1 Purpose

The monitoring subsystem is a separate runtime concern from the generated process execution graph. The generated handlers emit canonical lifecycle events, while the monitoring topology consumes those events and turns them into queryable operational views.

This separation matters because execution topics are transport-oriented. They show how work is routed. The monitoring topology instead answers questions such as:

- where is a specific process instance now,
- how many instances are currently in a given state,
- which activities are taking the longest,
- which instances appear stuck.

7.2 Streams topology

The core implementation lives in `org.gautelis.durga.monitoring.ProcessMonitoringTopology`. The topology consumes the shared `process-events` topic, filters out null keys or values, and then groups by process instance id.

From there, the monitoring flow is:

1. aggregate lifecycle events into a latest per-instance projection,
2. materialize that projection into a state store and a compacted topic,
3. regroup by current state key,
4. count instances per current state,
5. materialize those counts into a second state store and topic.

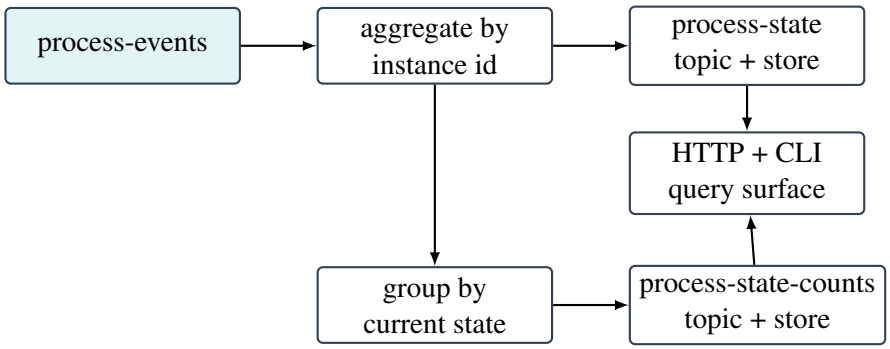

Figure 3: Durga monitoring topology from canonical lifecycle stream to queryable state

7.3 Materialized state

The per-instance projection is represented by `ProcessStateView`. It is not just a copy of the latest raw event. It accumulates enough information to support operational queries, including:

- current activity or lifecycle state,
- started, updated and completed timestamps,
- retry and failure information,
- per-activity timing information,
- correlation and business-key metadata.

That projection is written both to the Kafka Streams state store and to the `process-state` topic. The state store supports local low-latency queries. The topic keeps the projection observable and externalizable.

7.4 Counts, latency and stuck detection

The counts projection is simpler. The topology derives a state key from the latest per-instance view and counts how many instances currently map to each such key.

Latency and stuck-instance reporting are handled by the query service layer on top of the materialized state:

- latency summaries are computed from timestamps and durations recorded in `ProcessStateView`,
- stuck-instance detection scans state entries and compares their age against a threshold,
- process-specific counts are filtered from the global count view.

7.5 HTTP and CLI surface

The query service is exposed through `ProcessMonitoringHttpServer` and the companion CLI client. The main endpoints are:

- `/health`,
- `/instances/{processInstanceId}`,
- `/processes/{processId}/counts`,
- `/processes/{processId}/latency`,
- `/counts`,
- `/stuck`.

This is complemented by a lightweight dashboard page that polls the same query surface.

7.6 Relationship to demos

The monitoring topology is not only useful when a generated BPMN runtime is running. Repository-level demo publishers and scenario runners can emit synthetic lifecycle streams directly to `process-events`. That makes the monitoring subsystem independently testable and also makes it easier to understand the read-model behavior before a full generated process is involved.

7.7 Current limits

The topology is intentionally direct. It is strong enough for local experimentation and architectural validation, but it is not yet a comprehensive production operations subsystem. The main current limits are:

- the read model is local to the monitoring app instance,
- some queries are derived by scanning materialized state rather than by maintaining separate specialized indexes,
- history and trend reporting remain lighter than current-state reporting.

Even with those limits, the monitoring path is a major part of what makes Durga usable as an exploration tool rather than only a code generator.

8 Monitoring Interfaces

The monitoring subsystem is exposed through three interfaces that sit on top of the same Kafka Streams state:

- an HTTP query service,
- a small command-line client,
- a lightweight browser dashboard.

This split is intentional. The HTTP service is the primary integration surface, the CLI is the fastest way to inspect state from a terminal, and the dashboard is the fastest way to watch the system move.

8.1 HTTP service

The monitoring HTTP service is started by the monitoring application. Its main entry point is:

```
org.gautelis.durga.monitoring.ProcessMonitoringApp
```

The query layer is implemented by the repository classes `ProcessMonitoringQueryService` and `ProcessMonitoringHttpServer`.

The main endpoints are:

- `/health` for Kafka Streams readiness,
- `/instances/{processInstanceId}` for the latest instance view,
- `/processes/{processId}/counts` for process-specific current-state counts,
- `/processes/{processId}/latency` for activity latency summaries,
- `/counts` for the global state-count view,
- `/stuck` for stuck-instance detection.

The service is intentionally narrow. It is not trying to be a full workflow management API. Its role is to expose the read model produced by the monitoring topology.

8.2 CLI client

The command-line companion is `org.gautelis.durga.monitoring.ProcessMonitoringClient`. It exists because interactive inspection is common during development and BPMN exploration.

The main client operations are:

- `health`,
- `instance <processInstanceId>`,
- `counts [processId]`,
- `latency <processId>`,
- `stuck [processId] [olderThanSeconds]`.

The CLI is useful when the user is already working with the generated helper scripts or with local Kafka infrastructure from a terminal. It keeps the entire observation loop close to the generated runtime.

8.3 Dashboard

The browser dashboard is served from `/dashboard`. It polls the same HTTP endpoints that the CLI uses and presents:

- Kafka Streams health,
- counts by current state,
- latency summaries,
- stuck instances,
- selected instance details.

The dashboard is intentionally simple. It avoids a separate frontend build and keeps the monitoring surface close to the backend implementation. That makes it suitable for local architectural exploration, generated-process demos, and small-scale testing.

8.4 How the interfaces fit together

The three interfaces are not separate products. They are different views over the same materialized state:

1. generated or demo events are published to `process-events`,
2. Kafka Streams materializes state and counts,
3. the HTTP layer exposes that materialized state,
4. the CLI and dashboard consume the same HTTP surface.

This layering is important because it keeps the monitoring model coherent. The dashboard does not contain business logic that differs from the CLI, and the CLI does not bypass the query model. The read model remains single-sourced.

8.5 What this enables in practice

Taken together, the interfaces support a practical operator and learner workflow:

- publish a scenario or task outcome,
- watch `process-events`,
- query one instance in detail,
- inspect counts and latency summaries,
- switch to the dashboard when a broader view is helpful.

That workflow is one of the reasons Durga is usable as an exploration environment. Without these interfaces, the generated BPMN runtime would be much harder to evaluate as a concrete Kafka-based process system.

9 Generated Projects

9.1 Generated files

Each generated project includes more than Java sources. The generated output normally contains:

- Java runtime classes under `src/main/java/org/gautelis/durga/generated/`,
- a generated `pom.xml`,
- merged Kafka channel configuration in `application.yml`,
- a topic-creation script,
- sample payload data in `task-payloads.json`,
- a generated README,
- runnable helper scripts for demos, inputs, completions, failures, escalations and observers.

9.2 Helper scripts

The helper layer is part of the product surface, not just developer convenience. Generated projects include scripts such as:

- `demo-scenario.sh`,
- `send-task-input.sh`,
- `complete-task.sh`,
- `fail-task.sh`,
- `escalate-task.sh`,
- `complete-call-activity.sh`,
- `send-message-event.sh`,
- `send-signal-event.sh`,
- `watch-process-events.sh`,
- `watch-task-output.sh`.

That generated probe surface is important because it allows users to understand the runtime model without writing extra producers or Kafka CLI commands from scratch.

9.3 Development workflow

The intended workflow around a generated project is:

1. generate the project from a BPMN model,
2. inspect `summary.json`, topic config and helper scripts,
3. build the generated project independently,
4. run a starter or a scenario,
5. observe progress through `process-events` and the monitoring views,
6. adjust the BPMN model and regenerate.

This workflow is particularly useful when exploring BPMN-to-Kafka tradeoffs, because the generated runtime remains concrete and observable rather than abstract.

10 Generated Code Extension Strategy

Durga-generated code is meant to be a starting point, not an untouchable artifact. The safe engineering posture is:

- treat the BPMN model as the primary design source,
- treat generated code as scaffolding that is expected to be completed and adapted,
- keep custom logic easy to identify so regeneration does not become confusing.

10.1 What is usually edited

The most common edits in generated projects are:

- task business logic,
- XOR decision conditions,
- payload handling and validation,
- integration calls inside service-style workers,
- operational configuration values.

10.2 What should remain conceptually generated

The following areas are usually best kept close to the generated structure:

- topic and channel topology,
- lifecycle event emission shape,
- probe script surface,
- subprocess and boundary wiring,
- monitoring-facing event semantics.

10.3 Practical recommendation

In practice, the easiest way to keep a generated project healthy is to preserve the overall generated runtime structure while filling in task-specific behavior. If the BPMN model changes substantially, prefer regeneration and re-application of focused logic changes over ad hoc manual rewiring of the generated topology.

11 Generated Runtime Classes

Each generated project contains a set of Java classes under:

```
src/main/java/org/gautelis/durga/generated/
```

These classes are intentionally explicit. Durga does not try to hide the generated runtime behind one opaque engine class. Instead, it generates a visible set of handlers that map BPMN constructs onto Kafka consumers, producers and lifecycle events.

11.1 Class families

The generated classes usually fall into the following families:

- **Starter classes** start a new process instance by publishing the initial lifecycle event and the first task input.
- **Task services** handle BPMN tasks. Service-task-style handlers auto-complete after their work step, while user and manual task handlers enter a waiting state and require explicit completion, failure or escalation input.
- **Gateway services** handle XOR and parallel routing decisions, including fan-out and join behavior.
- **Event handlers** implement timer, message and signal behavior for both intermediate events and boundary-event reactions.
- **Call-activity helpers** model request and reply behavior for generated call activities.
- **Subprocess scope services** emit subprocess entry and completion lifecycle events and coordinate subprocess-local routing.
- **Process orchestrator** observes terminal flows and emits explicit process completion for the enclosing BPMN process.
- **Probe helpers** provide generated publishers and watchers that make the runtime easier to explore from the command line.

11.2 Task-oriented classes

Generated task services are the core execution units. Their responsibilities usually include:

- consuming a task input topic,
- emitting `ACTIVITY_ENTERED`,
- performing the generated task action or entering a wait state,
- emitting `ACTIVITY_COMPLETED`, `FAILED`, `ESCALATED` or `CANCELLED` when appropriate,
- forwarding the process to the next topic or boundary path.

This is one of Durga's central design choices: task execution and lifecycle reporting are generated together, rather than being split across unrelated runtime layers.

11.3 Routing and event classes

Gateway and event classes make control flow visible. XOR and parallel services turn BPMN branching into explicit topic routing. Timer, message and signal handlers make delays and external triggers concrete in the runtime graph.

Boundary handlers are especially important because they show where BPMN semantics extend beyond a straight sequence of tasks. Generated timer, error and escalation boundary classes observe task or scope state and branch into exceptional or reminder flows when their trigger conditions are met.

11.4 Scope-oriented classes

Subprocess support introduces generated scope services. These classes do more than route a single task. They make scope boundaries explicit by:

- emitting subprocess-level entered and completed events,
- mediating flow into and out of the subprocess graph,
- propagating failure, escalation and cancellation at scope level,
- supporting nested subprocess structure.

Event subprocesses extend this further. Their generated start and completion handlers model subprocess-internal event-driven branches without collapsing them into the main task path.

11.5 Probe and helper classes

Generated publishers, scenario runners and watchers are also part of the runtime class set. They are not part of the business flow itself, but they are part of how the generated system is meant to be explored. In practice, these helpers are often the fastest way to verify that a waiting state, boundary path or event subprocess behaves as expected.

11.6 Why the runtime is generated this way

The generated class surface is intentionally verbose. That is a tradeoff. It produces more classes than a hidden workflow engine would, but it also makes BPMN-to-Kafka mapping easier to inspect, test and discuss. Durga is therefore as much a scaffolding and exploration tool as it is a code generator.

12 Naming Conventions

Durga relies heavily on predictable names because the generated topology is intentionally visible. Naming is therefore part of the system design, not only a cosmetic detail.

12.1 Common naming patterns

The generator generally follows these patterns:

- process identifiers come from BPMN process ids,
- task topics use process and task identifiers,
- generated classes use process and BPMN element names converted to Java class names,
- helper scripts use short imperative file names such as `complete-task.sh`,
- monitoring topics use shared names such as:

```
process-events
process-state
process-state-counts
```

12.2 Why this matters

Consistent names make it easier to:

- inspect topic traffic in Kafka tools,
- map BPMN elements to generated Java handlers,
- reason about subprocess and boundary scope behavior,
- debug routing and completion behavior quickly.

12.3 Practical guidance

For the cleanest generated output, BPMN ids should be stable, readable and explicit. If BPMN models use unclear or heavily tool-generated ids, the generated Kafka and Java surface becomes harder to read as well.

13 Generated Project Lifecycle

Durga-generated projects are meant to be used iteratively rather than as one-off code dumps. The intended lifecycle is:

1. model a BPMN process,
2. scaffold a project from that model,
3. inspect generated topics, classes and helper scripts,
4. run demo flows or manual probes,
5. observe the lifecycle stream and monitoring views,
6. revise the BPMN model and regenerate.

13.1 Why regeneration matters

The BPMN model is still the primary design artifact. Generated Java and scripts are useful and often worth extending, but the center of gravity remains the process model. That is why the manual repeatedly treats regeneration as part of normal development rather than as an exception.

13.2 Two modes of use

There are two common ways to use a generated project:

- as a learning scaffold, where the main value is seeing how BPMN maps onto Kafka,
- as a starting point for a real process implementation, where the generated code becomes a base that is then extended, completed and hardened.

13.3 Practical implication

Because of that lifecycle, generated output needs to be readable, runnable and observable. Durga therefore treats helper scripts, summaries and monitoring support as first-class parts of the generated project, not as secondary extras.

14 Demo Flows And Helper Scripts

Durga is easier to understand when treated as something to run and probe, not only as a generator to inspect statically. For that reason, generated helper scripts are part of the intended product surface.

14.1 Purpose of the helper layer

The generated scripts serve three distinct roles:

- they provide runnable examples of the generated Kafka topology,
- they reduce the amount of manual Kafka producer and consumer work needed during exploration,
- they make BPMN semantics visible through explicit actions such as completion, failure, escalation and observation.

This matters because many workflow designs seem plausible in BPMN notation, but only become clear once concrete runtime events are observed. Durga therefore generates a probe layer alongside the executable runtime.

14.2 Common generated scripts

The exact set depends on the BPMN model, but the common script categories are:

- `demo-scenario.sh` for whole-process happy, stuck or failed runs,
- `send-task-input.sh` for manual injection into one task input topic,
- `complete-task.sh`, `fail-task.sh` and `escalate-task.sh` for human or exceptional task outcomes,
- `complete-call-activity.sh` for returning from a generated call activity,
- `send-message-event.sh` and `send-signal-event.sh` for external event stimulation,
- `watch-process-events.sh` and `watch-task-output.sh` for runtime observation.

14.3 Recommended learning loop

The most effective way to understand a generated process is usually:

1. generate a project from one small BPMN sample,
2. create the Kafka topics and start the generated runtime,
3. run `watch-process-events.sh`,
4. execute `demo-scenario.sh happy`,
5. repeat with a failure, escalation or timeout-oriented script,
6. inspect the monitoring endpoints or dashboard after each run.

This loop exposes the difference between nominal flow and exceptional flow with very little setup. It also makes one design principle visible: the execution topics carry work, but the canonical `process-events` stream carries the lifecycle truth.

14.4 A practical division of use

In practice the helper surface is most useful in three situations:

Learning A user wants to understand how one BPMN construct maps to Kafka consumers, producers and events.
Testing A developer wants to force a boundary, event-subprocess or call-activity path without creating a custom producer.
Debugging A generated process compiles, but the user needs to see where runtime routing or cancellation deviates from the BPMN intent.

14.5 Why this is documented in the manual

This documentation is deliberate. The helper scripts are not incidental wrappers around the runtime. They are part of how Durga explains itself. A BPMN-to-Kafka tool with no easy way to trigger and observe the generated process would be much harder to evaluate, especially when exploring tradeoffs around timers, boundaries, event subprocesses and subprocess scopes.

15 Configuration And Operations

15.1 Local runtime baseline

The normal local environment consists of:

- the bundled Kafka setup under `setup/`,
- generated topic configuration in the scaffolded project,
- the optional monitoring app and dashboard.

The repository is intentionally optimized for a local single-broker loop so the generated processes and monitoring projections can be explored quickly.

15.2 Important configuration surfaces

The main configuration locations are:

- repository-level runtime defaults in `src/main/resources/application.yml`,
- generated-project channel configuration in the scaffolded `application.yml`,
- local Kafka infrastructure in `setup/docker-compose.yml`,
- generated topic setup scripts in scaffolded output directories.

15.3 Operational expectations

In normal local startup:

- Kafka UI may log temporary connection failures before the broker is ready,
- generated runtimes rely on explicit topics and channels rather than implicit discovery,
- the monitoring app may report a transitional Streams state before its stores become queryable.

15.4 Where state lives

Durga uses several layers of state:

- transport history on execution topics,
- canonical lifecycle history on `process-events`,
- materialized latest state and counts in Kafka Streams stores and corresponding topics,
- local waiting and cancellation bookkeeping inside some generated handlers.

15.5 Recommended local operator loop

For practical local work:

1. start Kafka,

2. build the repository or generated project,
3. create topics,
4. run the generated process or a demo publisher,
5. start the monitoring app if query visibility is needed,
6. inspect lifecycle events, counts and latency.

16 Testing And Verification

The repository now has a meaningful regression net around the scaffolder and the monitoring path, even though it is still not a full end-to-end production validation environment.

16.1 Generator verification

Generator verification currently has three layers:

- dry-run tests that treat scaffold summaries and previews as a contract,
- generation tests that write selected projects to temporary directories and check the resulting files and configuration,
- compile-level tests that run Maven on representative generated projects.

This matters because the generator is split across BPMN parsing logic, StringTemplate groups and multiple helper generators. Simple unit tests are not enough on their own.

16.2 Representative sample coverage

The verification set intentionally spans several BPMN areas, including:

- message and signal events,
- timers and boundaries,
- subprocesses and nested subprocesses,
- call activities,
- event subprocesses.

16.3 Monitoring verification

The monitoring side is validated through:

- unit tests around the state-projection model,
- local startup and packaging checks,
- demo publishers and scenario runners that can drive the monitoring topology without a fully generated process runtime.

16.4 What is not fully covered yet

The remaining verification gaps are mostly at the integrated runtime level:

- full end-to-end process executions across all BPMN features,
- broader load or concurrency-oriented behavior,
- stronger operational testing of multi-instance monitoring scenarios.

For the current project phase, the test strategy is therefore strongest on generator correctness and local runtime exploration, and lighter on production-style operational proof.

17 Troubleshooting

Durga is now runnable enough that practical failure modes matter. The most common problems are not subtle generator bugs, but startup timing, missing topics, incomplete task logic, or the difference between execution state and monitoring state.

17.1 Kafka UI logs connection failures at startup

This is usually normal in the local setup. Kafka UI often starts before the broker is fully ready and logs transient connection failures. If the broker then comes up and the errors stop, this is not itself a Durga problem.

17.2 Monitoring app says the state store is not queryable yet

This normally means Kafka Streams has started but the state stores are not yet ready for queries. The correct response is usually to wait briefly and retry rather than to assume the topology is broken immediately.

17.3 A generated process compiles but does not advance

The most common causes are:

- topics were not created,
- the relevant generated application is not running,
- an XOR branch condition still contains placeholder logic,
- a user or manual task entered a waiting state and needs explicit completion,
- the process instance was cancelled by a boundary or interrupting event subprocess.

17.4 Boundary behavior does not appear to trigger

Check these points first:

- the triggering lifecycle event is actually emitted,
- the boundary is interrupting or non-interrupting as intended,
- the attached task or subprocess reached the expected state,
- failure or escalation was emitted through the generated helper path rather than by bypassing the lifecycle model.

17.5 An external completion arrives after cancellation

The generated runtime now suppresses much of this post-cancel progress, but late events can still be confusing during debugging. In such cases, the canonical `process-events` stream is the best source of truth: check whether cancellation happened before the late completion arrived.

17.6 Monitoring counts do not match raw topic intuition

This is usually a model misunderstanding rather than a bug. Execution topics contain history and routing traffic. The monitoring counts are derived from the latest projected instance state, which is the intended read model for “how many instances are in state X right now”.

17.7 The generated output looks harder to understand than expected

This often comes from BPMN naming. Unclear BPMN ids produce unclear Java classes, topics and helper artifacts. Cleaning up the BPMN model identifiers usually improves the generated runtime surface more than post-processing the generated files.

18 Known Design Tensions

Durga sits in a deliberate set of tensions. These are not accidental shortcomings; they are the result of choosing explicitness and inspectability over hiding complexity behind an engine.

18.1 Explicitness versus compactness

The generated runtime is larger and more verbose than a hidden workflow runtime would be. That cost is accepted because the system is easier to inspect, teach and debug when the BPMN-to-Kafka mapping is visible.

18.2 BPMN fidelity versus pragmatic Kafka mapping

Durga aims to support useful BPMN semantics, but it does so through explicit Kafka-oriented handlers and topics rather than through a full traditional BPMN engine. That means some semantics are intentionally pragmatic rather than exhaustive.

18.3 Regeneration versus local customization

Generated projects invite local edits, but regeneration remains important. This creates a normal tension between treating the output as disposable scaffolding and treating it as an evolving code base. The project currently resolves that by keeping the BPMN model primary and the generated runtime explicit.

18.4 Exploration versus production hardening

The repository is already strong enough for serious architectural exploration, but not every part of the system is optimized for hardened production operation. The current center of gravity remains clarity, runnable demos, and generator correctness.

19 BPMN Sample Catalog

The repository contains a growing set of BPMN models under `src/main/resources/bpmn`. They serve two purposes:

- they are regression inputs for the scaffolder and its tests,
- they are concrete learning models for understanding one BPMN feature at a time.

The samples are intentionally small. Most of them isolate one feature rather than trying to model a realistic end-to-end business process in full detail.

Model	Primary focus
`invoice_receipt.bpmn`	Baseline process with start, service work, review, approval or rejection, and terminal notification. This is the best first model to inspect.
`order_fulfillment.bpmn`	Legacy process sample kept as an additional reference model. It is useful for checking that generator changes are not too tightly coupled to the invoice-oriented examples.
`invoice_receipt_reminder.bpmn`	Intermediate timer catch event between normal process steps. Demonstrates delay-driven routing without a boundary event.
`invoice_message_exchange.bpmn`	Intermediate message throw and catch events. Shows explicit Kafka topic mappings for message-based collaboration.
`invoice_signal_exchange.bpmn`	Intermediate signal throw and catch events. Useful for contrasting signal semantics with the more direct message-event mapping.
`invoice_review_deadline.bpmn`	Interrupting timer boundary on a task. Demonstrates timeout handling and normal-flow suppression after deadline expiry.
`invoice_review_reminder_non_interrupting.bpmn`	Non-interrupting timer boundary on a task. Demonstrates reminder-style branching without cancelling the main task.
`invoice_processing_error.bpmn`	Interrupting error boundary on a task, driven by task failure events from the generated runtime.
`invoice_review_escalation.bpmn`	Interrupting escalation boundary on a task. Shows how escalation differs from failure as an explicit lifecycle outcome.
`invoice_call_activity.bpmn`	Call-activity request and reply mapping. Demonstrates the generated call and reply topics plus manual completion of the invoked activity.
`invoice_review_subprocess.bpmn`	Embedded subprocess with explicit entry and completion handlers. This is the basic scope-aware subprocess sample.
`invoice_nested_subprocess.bpmn`	Nested embedded subprocesses. Useful for understanding recursive scope generation and subprocess-specific input and output channels.

Model	Primary focus
`invoice_subprocess_deadline.bpmn`	Interrupting timer boundary attached to a subprocess scope. Demonstrates cancellation at the subprocess level rather than at one task.
`invoice_subprocess_reminder_non_interrupting.bpmn`	Non-interrupting timer boundary attached to a subprocess scope. Shows branch-without-cancel behavior for a whole scope.
`invoice_subprocess_error.bpmn`	Interrupting error boundary attached to a subprocess scope. Demonstrates subprocess-level failure propagation.
`invoice_event_subprocess_message.bpmn`	Non-interrupting embedded event subprocess with message start. Shows side-flow activation within an enclosing subprocess scope.
`invoice_event_subprocess_interrupting_message.bpmn`	Interrupting embedded event subprocess with message start. Demonstrates cancellation of the enclosing scope before the event subprocess branch starts.
`invoice_event_subprocess_timer.bpmn`	Embedded event subprocess with timer start. Demonstrates scoped delayed activation tied to the enclosing subprocess lifecycle.
`invoice_event_subprocess_error.bpmn`	Embedded event subprocess with error start. Demonstrates start-on-failure semantics driven by the canonical lifecycle stream.
`invoice_event_subprocess_escalation.bpmn`	Embedded event subprocess with escalation start. Demonstrates start-on-escalation semantics within a subprocess scope.

Taken together, the sample set functions as a compact executable coverage matrix. When a new BPMN function is added to the generator, adding a dedicated sample model alongside it has become the preferred way to document the feature, test the generator, and provide a runnable teaching example.

20 Example Walkthrough

20.1 Chosen example

The simplest useful walkthrough model in the repository is `src/main/resources/bpmn/invoice_receipt.bpmn`. It contains a start event, service work, a review step, an approval or rejection path, and a final notification step. That makes it a good reference model because it is small, but still representative of the core generator path.

20.2 Generation step

The normal way to generate the project is:

- `mvn-qcleanpackage`
- `java-jartarget/durga-1.0-SNAPSHOT.jar<bpmn-file>`

For this walkthrough, use the repository sample:

```
src/main/resources/bpmn/invoice_receipt.bpmn
```

By default the output is written under `generated/`. The generated project contains Java sources, a Maven build, Kafka topic declarations, helper scripts and a generated README.

20.3 What appears in the generated project

For a model like `invoice_receipt`, the generated output typically includes:

- worker services for automatic task execution,
- waiting-state handlers for any human-completion steps,
- a starter class that puts the first process event onto the start topic,
- a process orchestrator that detects terminal completions,
- helper publishers and watchers for demos and manual testing,
- generated Kafka bindings in `application.yml`.

The resulting project is intentionally explicit. Rather than hiding the process behind one entry point, Durga leaves the execution topics, task handlers and probes visible.

20.4 Runtime shape

For this example, the generated runtime follows a predictable pattern:

1. a starter publishes the initial process event,
2. generated task workers or waiting handlers consume task input topics,
3. each step emits lifecycle events to `process-events`,
4. routing services move the process to the next task or end event,

5. the orchestrator emits explicit process completion.

The same pattern scales outward to the richer BPMN samples in the repository. More advanced models add boundaries, subprocess scopes and event subprocesses, but they still rely on the same basic discipline: generated handlers move the process through Kafka topics while the canonical lifecycle stream captures the truth needed for monitoring.

20.5 How to explore the generated process

The generated helper scripts are the fastest way to understand the runtime:

- `demo-scenario.sh` exercises a whole process path,
- `send-task-input.sh` pushes one specific task input,
- `complete-task.sh`, `fail-task.sh` and `escalate-task.sh` let the user force specific task outcomes,
- `watch-process-events.sh` and `watch-task-output.sh` expose the generated flow as it runs.

This is a central idea in Durga. Generated projects are not only meant to compile; they are meant to be played with and observed.

21 Current Status

21.1 Maturity

Durga is in a strong experimental or prototype state rather than in a finished engine state. The generator is materially capable, the repository now has meaningful regression coverage, and the generated projects are usable for local exploration. At the same time, the project should still be understood as a controlled executable subset rather than a complete BPMN runtime.

21.2 What is solid today

Several parts of the system are now in reasonably good shape:

- the generator has been refactored away from a single large class and a single large template file,
- dry-run and generated-project compile tests cover representative BPMN features,
- the monitoring path is coherent and directly runnable,
- cancellation behavior for interrupting boundaries now affects runtime behavior rather than only the lifecycle stream,
- demo and probe tooling are generated as part of the process output.

21.3 Important limits

The present limits are mostly about semantics, not build hygiene:

- support is still a BPMN subset,
- some advanced scope semantics remain approximated rather than fully modeled,
- event subprocess support is deliberately scoped and not yet generalized to all top-level process cases,
- query and monitoring behavior are strong enough for local understanding, but not yet a complete production operations surface.

21.4 Likely next decisions

The next architectural decisions are not about whether the generator works. They are about where the support boundary should stop:

Top-level event subprocess semantics It remains an open design choice whether timer, error and escalation event subprocess starts should be given first-class process-level semantics.

Coverage breadth Inclusive and complex gateways remain unsupported.

Runtime fidelity There is still room to deepen true BPMN scope semantics if the project evolves from exploration tool toward a more complete runtime.

21.5 Practical summary

Durga is already useful as a technical exploration tool for BPMN-to-Kafka mapping. It is able to generate, run, observe and test a substantial set of workflow constructs locally. That is the right frame for the current

manual as well: this document should be read as system documentation for an actively evolving prototype with a real executable surface.

22 Glossary

BPMN Business Process Model and Notation, the process-modeling notation used as input to Durga.

Canonical lifecycle stream The shared `process-events` topic that carries normalized process and activity events.

Call activity A BPMN element used to model invocation of another process-like unit, mapped in Durga through explicit call and reply topics.

Event subprocess A subprocess triggered by an event within an enclosing scope instead of by the main sequence flow.

Execution topics The generated Kafka topics that move work between tasks, gateways and handlers.

Generated helper scripts The scaffolded scripts used to run demos, inject inputs, complete tasks, force failures and observe output.

Kafka Streams state store The local materialized store used by the monitoring app to answer low-latency queries.

Process orchestrator The generated runtime class that detects terminal flows and emits explicit completion for the enclosing process.

ProcessStateView The repository's materialized per-instance monitoring projection.

Scope cancellation The generated runtime behavior that suppresses or redirects normal flow after an interrupting boundary or interrupting event subprocess fires.

Scaffolder The BPMN-driven generator entry point, implemented by:

```
tools.org.gautelis.durga.BpmnScaffolder
```

Subprocess scope service A generated class that manages entry, completion and scope-level lifecycle behavior for an embedded subprocess.